

AI Usage & Academic Integrity Disclosure

Comprehensive Record of Generative AI Assistance & Independent Verification

Student Name:	Sankeerth Nara	Course:	Operating Systems (CS3.301 / OSN)
Project Title:	C-Shell (cshell) Implementation	Date:	August 25, 2026

1. Executive Summary & Philosophy of AI Use

This disclosure document provides a transparent report on the utilization of Generative Artificial Intelligence (AI) tools throughout the development lifecycle of the **C-Shell (cshell)** project. Generative AI tools were leveraged exclusively as an interactive technical sounding board, compiler diagnostic assistant, and algorithmic brainstorming partner. All system architecture decisions, low-level POSIX systems programming constructs, debugging syntheses, and test validation routines were conducted, audited, and confirmed independently.

2. Tools & Environments Consulted

- **AI Platforms / Models:** Google Gemini Large Language Model.
- **Primary Interaction Modalities:** Iterative conversational code debugging, architectural code modularization, edge-case analysis, and documentation structuring.
- **Target Environment:** Linux (Ubuntu / POSIX compliant), GCC compiler with strict flags (`-std=c2x -Wall -Wextra -Werror`).

3. Detailed Summary of AI Interactions

Domain / Component	AI Assistance Scope	Independent Student Work & Validation
Module Scaffolding & Headers builtin_*.h, executor.h	Provided boilerplate header structures, header guards, and function signature declarations.	Organized translation units, resolved cyclic dependencies, and customized global context structs.
Frecency Algorithm builtin_hop.c	Assisted in mathematical formulation for time-decayed scoring and ranking heuristic.	Implemented file I/O for <code>~/ .cshell_frecency</code> , resolved CWD-exclusion edge cases, and tuned exact-name priority weights.
File Traversal & Reversal builtin_peek.c, builtin_reveal.c	Brainstormed backward file reading strategies using lseek chunking and ASCII sorting helpers.	Coded chunk buffer manipulation, line-numbering flags (<code>-n, -r, -nr</code>), directory vs. file checks, and pipeline stream handlers.
Redirection & Piping Engine executor.c, parser.c	Advised on patterns for broadcast output redirection and multi-file input aggregation.	Engineered fork-exec pipelines, configured <code>dup2()</code> descriptors, closed unused pipe ends, and implemented asynchronous child process reaping via <code>waitpid(WNOHANG)</code> .
Compiler & Linker Diagnostics main.c, Makefile	Explained root causes of <code>-Werror=implicit-function-declaration</code> and undefined reference linker errors.	Synchronized cross-file function declarations, cleaned up unused variables, and verified strict zero-warning compilation.
Technical Documentation README.md	Structured sections and generated comprehensive Markdown formatting for features and specs.	Audited feature accuracy, verified command syntaxes, and removed superfluous testing sections.

4. Mathematical Formula Derived with AI

For the frecency-based navigation mechanism in hop, AI assisted in designing a smooth recency decay function without requiring external non-standard mathematical dependencies:

$$Score_{effective} = Score_{raw} / (1.0 + 0.05 \times Age_{hours})$$

This was integrated with a hierarchical priority multiplier: exact basename matches receive top weighting ($\times 10,000$), directory name substrings receive moderate weighting ($\times 100$), and ancestor path matches are assigned minimal weighting ($\times 0.001$).

5. Formal Student Declaration

I hereby declare that the work presented in this submission reflects my genuine understanding of Operating Systems principles and POSIX systems programming. The use of Generative AI was conducted transparently and constructively as detailed in this report, and at no point was raw unverified code integrated without thorough line-by-line inspection, compilation verification, and functional testing.

Sankeerth Nara

Student Signature

August 25, 2026

Date